

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
“ЛЭТИ” ИМ.В.И.УЛЬЯНОВА (ЛЕНИНА)»
КАФЕДРА МОЭВМ**

**ОТЧЕТ
по лабораторно-практической работе № 9
«Модульное тестирование приложения»
по дисциплине «Объектно - ориентированное программирование на
языке Java»**

Выполнил: Шарапов И.Д.

Факультет: КТИ

Группа: №3312

Подпись преподавателя: _____

Санкт-Петербург

2024

Содержание

Цель работы	3
Структура проекта.....	3
Перечень методов, которые тестируются.....	3
Выполнение тестов	4
Текст классов тестов	4
Приложение	5

Цель работы

Знакомство с технологией модульного тестирования Java- приложений с использованием системы JUnit.

Структура проекта

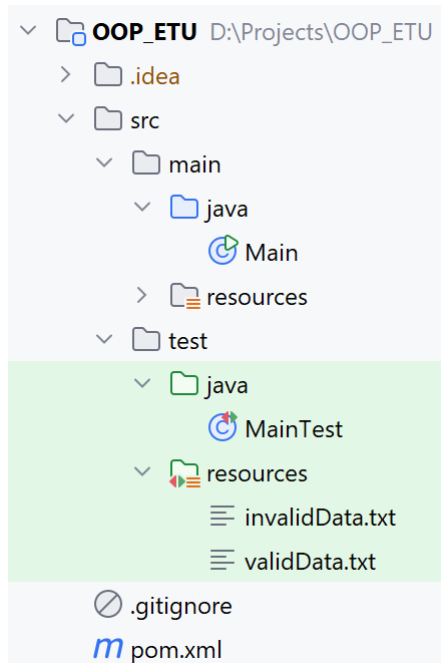


Рисунок 1 – Организация файлов в проекте

validData.txt					
1	Иванов Иван Иванович	A123BC77	15.03.2024	Превышение скорости	
2	Петров Петр Петрович	B456MN77	20.07.2023	Проезд на красный свет	
3					

Рисунок 2 – Правильный тестовый файл

invalidData.txt	
1	

Рисунок 3 – Тестовый файл с ошибкой (пустой)

Перечень методов, которые тестируются

- *saveDataToFile(File file)*
- *loadDataFromFile(File file)*

Выполнение тестов

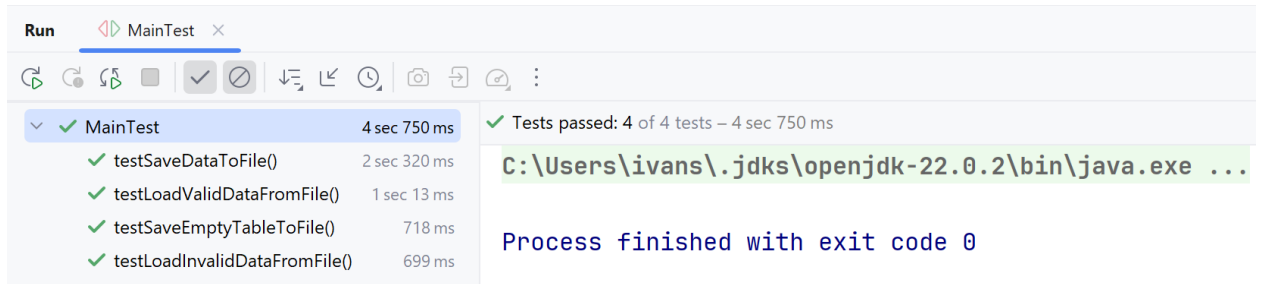


Рисунок 4 – Выполнение тестов

Текст классов тестов

Класс MainTest.java

```
import org.junit.jupiter.api.*;

import javax.swing.table.DefaultTableModel;
import java.io.*;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Тестовый класс для проверки методов класса {@link Main}.
 */
public class MainTest {

    private Main app;
    private DefaultTableModel testTableModel;

    /**
     * Конструктор по умолчанию для тестового класса {@link MainTest}.
     * Используется для создания экземпляров тестов.
     */
    public MainTest() {
        // Конструктор по умолчанию
    }

    /**
     * Инициализирует объект приложения и таблицу перед каждым тестом.
     */
    @BeforeEach
    public void setUp() {
        app = new Main();
        testTableModel = new DefaultTableModel(
            new String[]{"ФИО водителя", "Номер машины", "Дата нарушения", "Тип
нарушения"}, 0);
        app.tableModel = testTableModel;
    }

    /**
     * Тестирует метод {@link Main#saveDataToFile(File)}.
     * Проверяет, что данные из таблицы корректно сохраняются в файл.
     * @throws IOException если произошла ошибка ввода-вывода.
     */
    @Test
    public void testSaveDataToFile() throws IOException {
        // Создаем временный файл
        File tempFile = File.createTempFile("test", ".txt");
        tempFile.deleteOnExit();

        // Добавляем данные в таблицу
        testTableModel.addRow(new Object[]{"Иванов Иван Иванович", "A123BC77",
"15.03.2024", "Превышение скорости"});
        app.saveDataToFile(tempFile);

        // Проверяем содержимое файла
        try (BufferedReader reader = new BufferedReader(new FileReader(tempFile))) {
```

```

        String firstLine = reader.readLine();
        assertNotNull(firstLine);
        assertEquals("Иванов Иван Иванович\tA123BC77\t15.03.2024\tПревышение скорости",
firstLine.trim());
    }
}

/**
 * Тестирует метод {@link Main#loadDataFromFile(File)} с корректными данными.
 * Проверяет, что данные из файла правильно загружаются в таблицу.
 *
 * @throws IOException если произошла ошибка ввода-вывода.
 */
@Test
public void testLoadValidDataFromFile() throws IOException {
    // Загружаем корректный файл
    File validFile = new
File(getClass().getClassLoader().getResource("validData.txt").getFile());
    app.loadDataFromFile(validFile);

    // Проверяем, что данные загружены в таблицу
    assertEquals(2, testTableModel.getRowCount());
    assertEquals("Иванов Иван Иванович", testTableModel.getValueAt(0, 0));
    assertEquals("A123BC77", testTableModel.getValueAt(0, 1));
    assertEquals("15.03.2024", testTableModel.getValueAt(0, 2));
    assertEquals("Превышение скорости", testTableModel.getValueAt(0, 3));
}

/**
 * Тестирует метод {@link Main#loadDataFromFile(File)} с некорректными данными.
 * Проверяет, что некорректный файл обрабатывается без ошибок.
 */
@Test
public void testLoadInvalidDataFromFile() {
    // Загружаем некорректный файл
    File invalidFile = new
File(getClass().getClassLoader().getResource("invalidData.txt").getFile());
    assertDoesNotThrow(() -> app.loadDataFromFile(invalidFile));

    // Проверяем, что таблица пустая
    assertEquals(0, testTableModel.getRowCount());
}

/**
 * Тестирует метод {@link Main#saveDataToFile(File)} с пустой таблицей.
 * Проверяет, что пустая таблица корректно сохраняется в файл.
 *
 * @throws IOException если произошла ошибка ввода-вывода.
 */
@Test
public void testSaveEmptyTableToFile() throws IOException {
    // Создаем временный файл
    File tempFile = File.createTempFile("test", ".txt");
    tempFile.deleteOnExit();

    // Сохраняем пустую таблицу
    app.saveDataToFile(tempFile);

    // Проверяем, что файл пуст
    try (BufferedReader reader = new BufferedReader(new FileReader(tempFile))) {
        assertNull(reader.readLine());
    }
}
}

```

Приложение

Ссылка на репозиторий:

https://github.com/DexTver/OOP_ETU/tree/lab_09